



Deployment — Đưa Agent Lên Cloud

AICB-P1 · Ngày 12 · Từ localhost đến production URL

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

*“Bạn demo cho sếp thấy agent chạy trên laptop.
Sếp hỏi: khi nào 100 người dùng được? —
và liệu nó có ngốn hết ngân sách không?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Từ localhost đến production
2. **Agent vs deploy truyền thống**
3. Docker & containerization (2026)
4. Thách thức riêng của agent
5. **Agent chạy ở đâu:
server/client/on-device**
6. Cloud options + managed runtimes (Tier 0)
7. Hosting MCP servers
8. API gateway & security
9. Scaling + frontier-scale serving
10. CI/CD & eval gates
11. **Nâng cao: production-grade (tùy chọn)**
12. Checklist + phụ lục lệnh/code
13. Lab 12 + preview Day 13

- Hiểu **gap giữa dev và production**: dependencies, config, secrets, networking
- Viết **Dockerfile** hiện đại (multi-stage + uv + slim/distroless) để đóng gói agent
- Nắm **3 thứ agent phá vỡ web infra thông thường**: long-running, stateful, cost
- So sánh **cloud options** theo trục quan trọng nhất với agent: **request timeout**
- Biết về **managed agent runtimes** mới (Bedrock AgentCore, Vertex Agent Engine)
- Thiết kế **API gateway** + cost protection và deploy agent có **public URL** hoạt động

Agent đã được containerize và deploy lên cloud, có health check endpoint, basic authentication, cost guard, và accessible qua public URL

- 1 Dockerfile (multi-stage, uv, <500MB) + docker-compose cho agent + dependencies
- 1 deployed instance trên Railway hoặc Render
- 1 health check endpoint (/health) + streaming endpoint (SSE)
- 1 public URL mà bất kỳ ai cũng có thể truy cập và dùng agent

Từ Localhost Đến Production

Agent chạy trên máy mình khác rất xa với agent chạy cho 100 người — gap đó không chỉ là “copy code lên server”

11 ngày đã build

- LLM API + prompt engineering
- RAG pipeline grounded
- Multi-agent + MCP
- UX + trust layer
- Guardrails + safety

Nhưng đang chạy trên

- localhost:8000
- API keys trong .env file
- Chỉ 1 user (chính mình)
- Không health check
- Tắt laptop = agent chết

Lưu ý: “It works on my machine” là câu nói nổi tiếng nhất trong lịch sử software engineering. Day 12 giải quyết đúng vấn đề này.

Khía cạnh	Dev (localhost)	Production
Dependencies Config	“pip install” thủ công .env file trên máy	Đóng gói cùng container Environment variables, secrets manager
Networking Users	localhost:8000 1 (chính mình)	HTTPS, domain, load balancer N users đồng thời
Failure	Restart thủ công	Auto-restart, health check

Environment parity: dev/staging/prod **càng giống nhau càng ít bug** khi deploy.

Một CRUD app trả lời trong $<1s$. Agent thì khác về bản chất — và đó là nguồn gốc của mọi thách thức deploy hôm nay.

Reasoning loop chạy 10–60s+ (có khi vài phút). **Phá vỡ** timeout 29–60s của gateway/proxy.

Có conversation memory + tool history. **Mâu thuẫn** với quy tắc “stateless process” của 12-factor.

Mỗi call gửi lại cả history → cost tăng **siêu tuyến tính** (50–1000× token so với chat).

Lưu ý: Giữ 3 tính chất này trong đầu suốt cả bài. Mỗi section sau giải quyết một hệ quả của chúng.

1. **Config in env:** không hardcode API keys
2. **Stateless processes:** agent không giữ state trên instance
3. **Port binding:** export service via port
4. **Dev/prod parity:** giữ gap nhỏ nhất

- ☐ Secrets management
- ☐ Health check endpoint
- ☐ Structured logging
- ☐ Monitoring endpoint
- ☐ Graceful shutdown

Lưu ý: Factor VI (stateless) nói rõ: “không dùng sticky session”. Nhưng agent có memory — ta sẽ giải quyết bằng cách *externalize state* (mục *Thách Thức Riêng Của Agent*), không phải bỏ qua nguyên tắc.

Agent Deploy vs Deploy Phần Mềm Truyền Thống

Tin tốt: bạn ship agent bằng **đúng cỗ máy** đã có (CI/CD, container, load balancer). Tin cần nhớ: có 3 thứ bị *định nghĩa lại* — và đó là nơi agent khác biệt

Giữ nguyên (đừng phát minh lại)

- CI/CD pipeline, build artifact
- Immutable image, canary/rolling
- IaC (Terraform)
- Load balancer, health check
- 12-factor, stateless web tier

Bị định nghĩa lại (cái mới)

- **Test:** eval gate, không phải exact-match
- **Hoá đơn:** token runtime, không CPU-hour
- **Dependency:** model ngoài, rate-limit, deprecate
- **State:** hội thoại, không DB row
- **GPU economics:** không CPU/RAM

Cùng cái hộp; **cái bên trong** và **cách quyết định “đạt”** mới là phần khác.

	App CRUD truyền thống	AI Agent
Kiểm thử	Unit test, assert chính xác	Eval gate (golden set, LLM-judge), gate theo điểm
Chi phí	CPU/RAM-hour, đoán trước được	Token/request , chỉ biết sau khi chạy
Latency	Mili-giây, đồng bộ	Giây–phút, streaming (SSE) + async
State	DB rows	Hội thoại / memory (checkpointer)
Dependency	Bạn kiểm soát & tự version	Model bên ngoài : rate-limit, bị deprecate
Scaling unit	CPU/RAM	GPU + VRAM

Lưu ý: Luận điểm: dùng **cùng machinery** (CI/CD, canary, immutable image, IaC, LB) — nhưng định nghĩa lại **the test, the bill, the dependency**.

Khác thư viện bạn pin trong `requirements.txt`: model sống trên server của người khác, có thể bị **khai tử** và có **trần thông lượng** do nhà cung cấp đặt.

- Pin **model ID** như một deploy artifact (cùng prompt version)
- Provider báo trước ≥ 60 ngày rồi **request fail**
- Thực tế: Sonnet 3.5 (2025), Opus 3 (1/2026), gpt-4o (3/2026)
- Tắt auto-upgrade để rollback an toàn

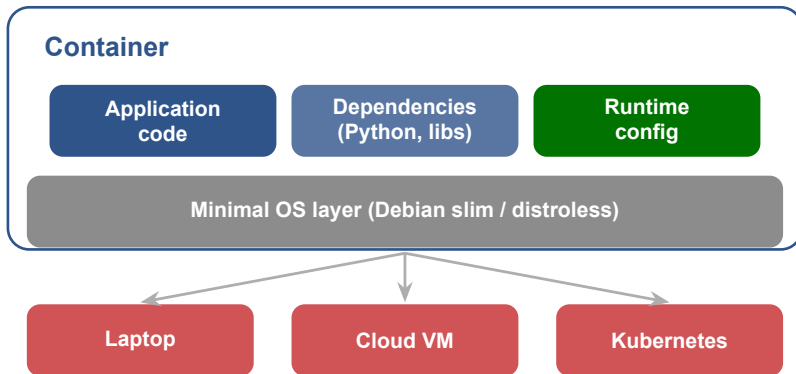
- RPM / TPM theo **tier** của nhà cung cấp
- VD GPT-4o: Tier 1 = 500 RPM/200k TPM $\rightarrow$ Tier 5 = 10k/30M
- Trần do **vendor** đặt, không phải autoscaler của bạn
- $\rightarrow$ retry/backoff + nhiều key (mục Scaling)

Lưu ý: Lập kế hoạch deploy phải tính lịch **khai tử model** của nhà cung cấp — nó là một forcing function lên release calendar của bạn.

Docker — Đóng Gói Agent Thành Container

Container giải quyết “it works on my machine” bằng cách đóng gói mọi thứ agent cần thành 1 unit chạy được ở bất kỳ đâu

03



Ý chính: Container = app + deps + runtime đóng gói thành 1 unit. **Build 1 lần, chạy ở mọi nơi.**


```
# Stage 1: build deps with uv (Rust-fast, ~10x pip)
FROM python:3.12-slim AS builder
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy
RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    uv sync --locked --no-install-project --no-dev

# Stage 2: slim runtime
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY . .
ENV PATH="/app/.venv/bin:$PATH"
RUN useradd -m app && chown -R app /app
USER app # non-root
CMD ["fastapi", "run", "main.py", "--host", "0.0.0.0"]
```

Lưu ý: Target <500MB: uv+cache, --locked, non-root, .dockerignore.

Base image	Size (đĩa)	Ghi chú cho AI agent
python:3.12	~1.0 GB	Full — thừa toolchain, attack surface lớn
python:3.12-slim	~150 MB	Default tốt cho ML Python (glibc, manylinux wheels)
distroless	~66 MB	Gọn nhất + an toàn; không shell (debug khó)
python:3.12-alpine	~55 MB	TRÁNH cho ML — xem cảnh báo

Lưu ý: Alpine dùng musl libc, không tương thích manylinux wheels (glibc) → pip **compile từ source** mọi package. Thực đo (pandas+matplotlib): slim 30s/363MB vs Alpine 26 phút/851MB = ~**52×** chậm, lớn hơn.

1. **Scan CVE:** Trivy / Docker Scout / Gype trước khi deploy
2. **Non-root:** USER app, không chạy root trong container
3. **Pin digest:** FROM ...@sha256:... thay vì tag mềm

Software Bill of Materials (SPDX/CycloneDX): liệt kê mọi package trong image.

Sinh bằng `syft / docker buildx --sbom`.

Giờ là **yêu cầu pháp lý** (US EO 14028, EU Cyber Resilience Act).

Lưu ý: Distroless ít CVE hơn nhưng **không phải zero** — runtime và C libs vẫn có. Scan là việc lặp lại, không phải một lần.

Agent stack điển hình

- **Agent service:** FastAPI + LLM logic
- **Vector store:** Qdrant (6333/6334)
- **Cache:** Redis (6379) cho session/rate limit
- **Reverse proxy:** Nginx (optional)

- `docker compose up` (V2, không gạch nối)
- Bỏ key `version`: (đã obsolete)
- `depends_on`: `condition: service_healthy`
- Service gọi nhau bằng tên (DNS): `qdrant:6333`

Bắt đầu với 2 services: **agent + vector store**. Thêm Redis và Nginx khi hệ thống cần scale.

```
from fastapi import FastAPI
from sse_starlette.sse import EventSourceResponse

app = FastAPI()

@app.get("/healthz")           # health check for LB / Cloud Run
def healthz():
    return {"status": "ok"}

@app.post("/chat")            # streaming is the default, not the exception
async def chat(req: ChatRequest):
    async def gen():
        async for chunk in run_agent(req.messages):
            yield {"event": "token", "data": chunk}
        yield {"event": "done", "data": "[DONE]"}
    return EventSourceResponse(gen())
```

Lưu ý: /healthz cho hạ tầng biết agent sống; /chat **stream** token qua SSE — cải thiện UX + lách timeout.

Thách Thức Riêng Của Agent

Long-running + stateful = web infra thông thường “gãy”. Đây là phần mà một bài deploy bình thường bỏ qua — nhưng agent thì không thể

Reasoning loop của agent thường lâu hơn timeout mặc định của hạ tầng. Request bị cắt giữa chừng → user thấy lỗi 504.

Hạ tầng	Timeout mặc định	Ghi chú
AWS API Gateway	29s → 504	Có thể nâng (từ 6/2024)
Heroku router	30s (initial byte)	Không chỉnh được
AWS ALB (idle)	60s	Cắt stream im lặng
nginx proxy_read_timeout	60s	Giữa 2 lần đọc
Fly.io proxy (idle)	60s	Streaming reset timer
Railway public HTTP	15 phút	Private network: vô hạn
Google Cloud Run	60 phút (max)	Phù hợp agent

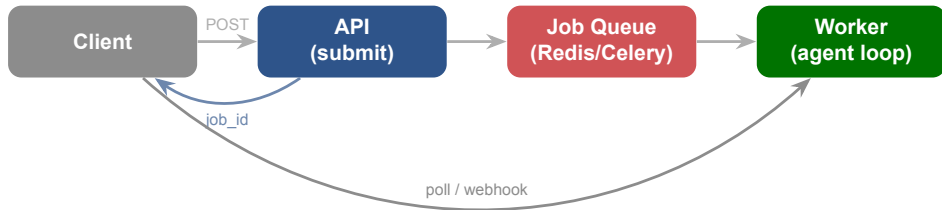
Lưu ý: 2 cách vượt: (1) **stream** từng token (giữ connection “sống”); (2) route long calls qua **private network** hoặc chuyển sang **async job**.

- Token streaming là **một chiều** (server→client)
- Chạy trên HTTP/1.1 thường, không cần upgrade
- EventSource tự reconnect + Last-Event-ID
- OpenAI & Anthropic đều dùng SSE (stream:true)

Lưu ý: 2 cái bẫy proxy hay gặp:

- nginx proxy_buffering on (mặc định) gom token lại → tắt bằng proxy_buffering off hoặc header X-Accel-Buffering: no
- Agent “suy nghĩ” im lặng >60s → gửi **heartbeat** : ping mỗi ~15s để không bị cắt idle

Stream byte đầu tiên **trong vòng 30s** → đa số gateway giữ connection mở cho cả response dài. Streaming vừa cải thiện UX vừa là cách lách timeout.



- **Submit-and-poll:** API trả `job_id` ngay, client hỏi kết quả sau (hoặc nhận webhook)
- Tool: **Celery** (broker), **RQ** (Redis), **Cloud Tasks** (managed)
- Batch lớn không gấp? **Batch API** của OpenAI/Anthropic: rẻ hơn **50%**, trả trong 24h

12-factor nói agent phải stateless để scale. Nhưng agent có memory. Giải pháp: **externalize state**, không giữ trên instance.

- Conversation/session → **Redis / Postgres**
- LangGraph **checkpoint**er (PostgresSaver) keyed bằng thread_id
- Bất kỳ instance nào cũng phục vụ được request → scale tự do

Cho agent chạy nhiều bước/nhiều ngày, resume sau crash:

- **Temporal** (replay event history)
- **Inngest** (step memoization)
- **LangGraph Platform** (managed persistence)

Lưu ý: “Sticky session” (session affinity) chỉ là **best-effort** — Cloud Run phá vỡ nó khi instance bị kill hay quá tải. Đừng phụ thuộc vào nó; externalize state mới chắc.

Mỗi request agent giữ worker **rất lâu** (đợi LLM/tool) → dễ cạn worker pool.

- `async def` cho 1 worker phục vụ nhiều request khi đang `await I/O`
- **Bẫy:** blocking call trong `async def` làm **ngheñ cả event loop**
- Cloud Run concurrency: mặc định 80, max 1000/instance

Load model/embeddings lúc khởi động → cold start chậm.

- **Min instances** (Cloud Run) giữ instance ấm
- **Provisioned concurrency** (Lambda)
- **Lazy-load** object hiếm dùng ra khỏi cold path

Agent **I/O-bound** (đợi LLM) → tăng concurrency tiết kiệm instance; giữ blocking code trong `def` thường để FastAPI offload sang threadpool.

Agent Chạy Ở Đâu? Server / Client / On-Device

Một trục kiến trúc deploy mà nhiều người bỏ qua: vòng lặp agent + tools + API key thực sự **chạy ở đâu**? Câu trả lời quyết định bảo mật, chi phí, và quyền riêng tư

Vị trí	Loop chạy ở	API key	Cost/token	Năng lực
Server-side	Backend của bạn	An toàn (server)	Bạn trả hết	Frontier
Client (chỉ UI)	Trình duyệt, call qua proxy	Vẫn cần proxy	Bạn trả hết	Frontier
In-browser model	Trình duyệt (WebGPU)	Không cần	Bằng 0	~1–3B
On-device	NPU/GPU thiết bị	Không cần	Bằng 0	~3B
Edge	Node biên (Workers AI)	Ở provider	Pay-per-call	~7B+
Hybrid	On-device → escalate cloud	Cloud tin cậy	Rẻ→đắt	Nhỏ→frontier

Đa số agent production là **server-side**: loop + tools + key ở backend, client chỉ là lớp mỏng gửi message. Đổi lại: mọi token đi vòng qua server bạn, bạn trả toàn bộ compute.



key thêm ở đây,
KHÔNG bao giờ xuống browser

Lưu ý: Build tool **nhúng** biến `VITE_/NEXT_PUBLIC_` thẳng vào JS bundle → key ship xuống trình duyệt và **bị lấy cắp**. Vì vậy ngay cả agent “client-side” với model frontier **vẫn cần backend proxy** (Backend-for-Frontend) + rate limit. Chỉ **on-device / in-browser model** mới thật sự keyless.

- **Apple Foundation Models** (~3B, on-device, offline, free)
- **Chrome Gemini Nano** / Prompt API (Chrome 138, không gửi data đi)
- **MS Phi Silica** (NPU, Copilot+ PC)
- **WebLLM / transformers.js** (WebGPU trong browser)

Cloudflare Workers AI: inference trên GPU ở 200+ thành phố, pay-per-call, OpenAI-SDK compatible.

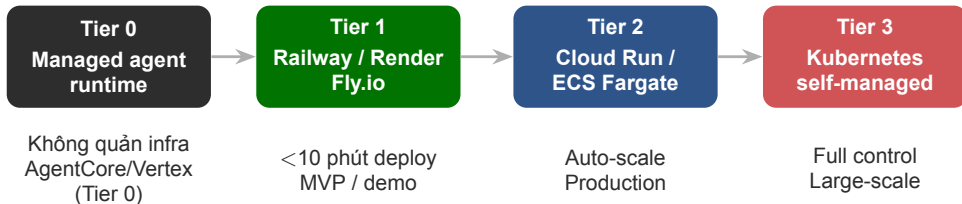
Đánh đổi: privacy + offline + cost 0, nhưng năng lực ~1–3B, tốn pin, và **cold load** weights lần đầu.

Model nhỏ on-device lo việc thường → **escalate** lên model lớn trên cloud khi khó.
VD Apple: ~3B on-device → **Private Cloud Compute** (giữ nguyên cam kết privacy trên cloud). Tiêu chí định tuyến thường *không công bố*.

Cloud Deployment Options

Không có 1 platform đúng cho mọi trường hợp — và với agent, trực quan trọng nhất là **request timeout**, không phải giá

06



Bắt đầu Tier 1 (Railway/Render). Hiểu flow deploy trước, migrate lên Tier 2/3 khi business cần, hoặc Tier 0 nếu muốn bỏ qua việc quản hạ tầng.

Platform	Max request/runtime	Scale-to-0	GPU	Agent fit
Railway	15 phút / ∞ private	Không	Không	OK (route nội bộ)
Render	~100 phút	Free tier	Không	OK
Fly.io	60s idle (stream reset)	Có	Có	OK + streaming
Cloud Run	60 phút	Có (cả GPU)	L4	Mạnh
AWS App Runner	~120s	Provisioned	Không	Deprecated
ECS Fargate	Không giới hạn	Không	—	Mạnh (always-on)
Modal	24 giờ	Có (snapshot)	H100/A100	Mạnh (GPU)
Vercel functions	5 phút (Hobby)	Có	Không	Kém

Lưu ý: Cập nhật 2026: Railway **bỏ free tier** (giờ \$5 trial credit); AWS App Runner **deprecated** (Mar 2026) → ECS Express Mode; Cloud Run GPU **GA** (6/2025, scale-to-zero, ~5s start).

Vercel / Lambda functions

- Hard cap thời gian: Vercel Hobby **5 phút**, Pro 13 phút → 504
- Body cap 4.5 MB
- Stateless — mất context giữa các invoke

Container-based hợp hơn

- Cloud Run 60 phút, Fargate vô hạn
- Giữ được connection cho streaming dài
- Min-instance giữ ấm, tránh cold start

Lưu ý: Câu cũ “serverless cold start 5–15s” giờ **lỗi thời**: Cloud Run GPU start ~5s, Modal snapshot nhanh ~10×. Vấn đề thật của serverless với agent là **timeout cap**, không phải cold start.

Các bước

1. Kết nối GitHub repo
2. Railway tự detect Dockerfile (builder Railpack 2026)
3. Set environment variables
4. Click Deploy
5. Nhận public URL

- Auto-detect Dockerfile
- Environment variables UI
- Custom domain + SSL miễn phí
- Logs real-time
- \$5 trial credit (đủ cho lab)

Lưu ý: Hết free tier: Railway giờ tính theo usage, new user có \$5 credit dùng thử (không cần thẻ). Bind đúng `0.0.0.0:$PORT` — Railway inject PORT tự động.

Managed Agent Runtimes (Tier 0)

Danh mục mới hẵn của 2025–26: deploy agent mà **không phải quản container** — runtime, memory, identity đều managed

Khác container PaaS: bạn không viết Dockerfile/lo scaling. Platform cung cấp runtime + session + memory + identity sẵn, thường tính theo tiêu thụ.

Sản phẩm	GA	Điểm nhấn
AWS Bedrock Agent-Core	Oct 2025	8 giờ/session , microVM cô lập mỗi session
Vertex AI Agent Engine	Mar 2025	Sessions + Memory Bank (long-term memory)
Azure AI Foundry Agent	May 2025	No-code + hosted; miễn phí service (trả token)
OpenAI AgentKit	Oct 2025	Agent Builder + ChatKit (UI nhúng)

Framework-agnostic (LangGraph, CrewAI, ADK, OpenAI Agents SDK...), hỗ trợ MCP/A2A, session isolation, và idle/I-O-wait thường **không tính tiền**.

Chọn Tier 0 khi

- Muốn ship nhanh, không có team infra
- Cần session isolation + memory sẵn
- Agent chạy rất lâu (AgentCore 8h)
- Đã ở sẵn hệ sinh thái AWS/GCP/Azure

Tự deploy (Tier 1–3) khi

- Cần kiểm soát đầy đủ stack
- Tránh vendor lock-in
- Tối ưu cost ở quy mô lớn
- Yêu cầu compliance/networking riêng

Lưu ý: Cho lab 12, ta vẫn **tự containerize + deploy** (Tier 1) để hiểu cơ chế. Tier 0 là lựa chọn production khi không muốn quản hạ tầng — biết là đủ.

Hosting MCP Servers

Day 9 dạy agent gọi MCP tools. Khi MCP server cần chạy *re-mote* cho nhiều client, nó cũng là một service phải deploy — và phải bảo mật đúng

08

Client **spawn server làm subprocess**, nói chuyện qua stdin/stdout.

- Chạy cùng máy với agent
- Không cần OAuth — lấy credential từ env
- Hợp dev / desktop

Một endpoint (vd /mcp), POST+GET, SSE *bên trong*.

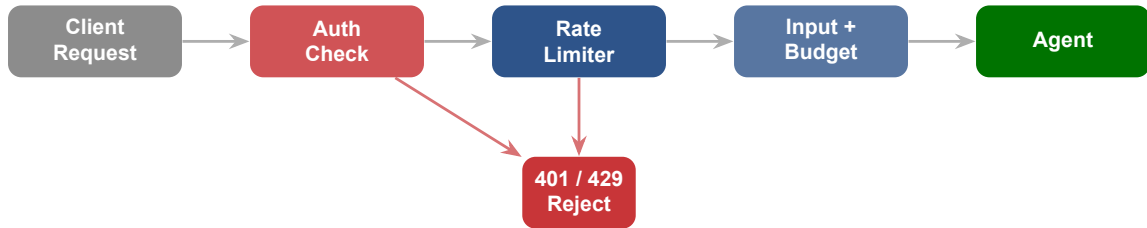
- Process độc lập, phục vụ nhiều client
- **Thay thế** transport HTTP+SSE cũ (2024-11-05)
- Phải host + bảo mật như API thật

Lưu ý: Remote MCP server (spec 2025-06-18) phải dùng **OAuth 2.1**: client bắt buộc PKCE; server đóng vai **Resource Server** (RFC 9728) và phải xác minh token được cấp *đúng cho nó* (chống “confused deputy”). stdio thì KHÔNG cần OAuth.

API Gateway & Security

Agent trên cloud cần lớp bảo vệ trước khi request đến logic — authentication, rate limiting, và **cost protection** (nơi nhiều startup đã “cháy túi”)

09



Mỗi request phải qua **auth** → **rate limit** → **validate + budget check** trước khi agent xử lý. Reject sớm = tiết kiệm tokens và tiền.

Đơn giản nhất.

Header: X-API-Key

Dùng khi: internal, MVP,
B2B (M2M)

Stateless auth.

Bearer token + expiry

Dùng khi: user-facing
app, microservices

Delegated auth + PKCE.

Chuẩn cho MCP/agent
remote

Dùng khi: platform, re-
mote MCP

Lưu ý: Cho MVP: **API key** là đủ. Đừng over-engineer auth trước khi có user thật.
Nhưng nếu host **remote MCP server**, OAuth 2.1 là bắt buộc theo spec.

Rủi ro (có thật)

- **\$47.000 trong 11 ngày:** 4 agent retry vô hạn, có log nhưng không có hard limit
- **\$96.000 Vercel:** app Cara tăng 100k→900k user trong vài ngày
- Prompt injection → token explosion
- API key bị lộ → hacker xài

Bảo vệ

- **Pre-call admission control:** check budget còn lại *trước khi* gọi LLM, từ chối nếu vượt
- **Per-tenant budget:** key theo (tenant, workload, model)
- **Rate limiting:** token bucket (rate + burst)
- **Circuit breaker:** tắt khi anomaly

Lưu ý: Bẫy lớn: ngân sách “monthly budget” của OpenAI là **thông báo, KHÔNG chặn chi tiêu**. Hard cap thực sự là **per-project** limit. Alert của provider là *phản ứng sau* → tự build admission control *chặn trước*.

Security basics

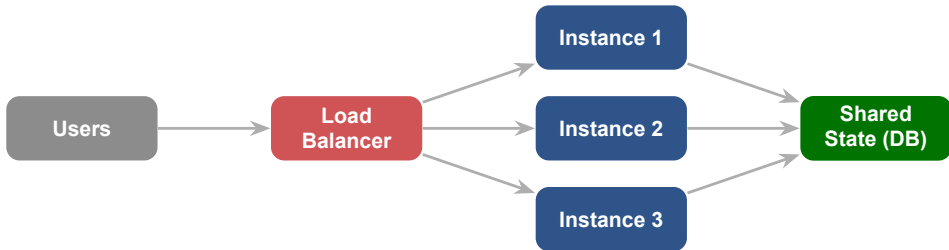
- **HTTPS:** Railway/Render cấp SSL tự động
 - **CORS:** chỉ là kiểm soát *trình duyệt* — KHÔNG phải authz, API vẫn cần auth
 - **Secrets:** dùng secret manager (AWS Secrets Manager, Doppler, Infisical) thay `.env` — có rotation + audit
 - **GitHub:** push protection bật mặc định; key lộ bị Anthropic/OpenAI auto-revoke
- LLM01 Prompt Injection
 - LLM02 Sensitive Info Disclosure
 - LLM06 Excessive Agency
 - **LLM10 Unbounded Consumption** (cost/DoS)

Lưu ý: Kiểm tra ngay: `.env` có trong `.gitignore`? Key có lỡ commit lên GitHub? Nếu có → **revoke và tạo key mới ngay lập tức.**

Scaling & Reliability

Agent MVP không cần Kubernetes — nhưng cần hiểu cơ bản về scaling và reliability để hệ thống không chết khi có nhiều user hơn

10



Lưu ý: Agent là **I/O-bound**: một instance có thể đầy request đang chờ LLM mà CPU vẫn thấp. → Autoscale theo **concurrency / queue depth** (Knative, KEDA theo tín hiệu vLLM `num_requests_running`, Ray Serve `target_ongoing_requests`), không theo CPU. Điều kiện tiên quyết: agent **stateless**, state ở DB/Redis.

“Còn sống?” Fail → **restart** container.
GET /health

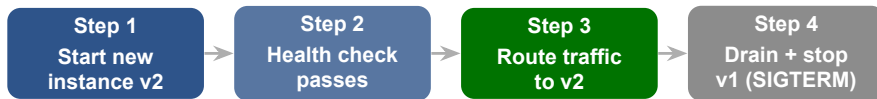
“Sẵn sàng nhận request?” Fail → **gỡ khỏi LB** (ngưng traffic), **KHÔNG** restart

Cho boot chậm (load weights). Gate live-ness/readiness đến khi pass

GET /health trả về:

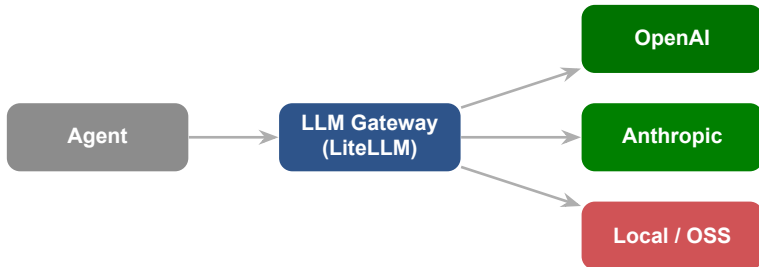
- status: “ok” / “degraded”
- uptime: seconds
- version: app version
- dependencies: DB, vector store, LLM reachable?

Lưu ý: **Liveness** restart, **readiness** chỉ ngắt traffic — cấu hình nhằm readiness thành liveness = restart loop vô ích.



Lưu ý: Graceful shutdown cho agent: khi nhận SIGTERM, phải **drain** request đang chạy dở (một agent turn có thể dài). Đặt `terminationGracePeriodSeconds` **lớn hơn worst-case agent turn**, nếu không request bị cắt giữa chừng.

Canary/blue-green với **Argo Rollouts**: shift traffic dần + `AnalysisRun` tự rollback nếu metric xấu. Railway/Render hỗ trợ rolling sẵn.



- **1 endpoint, format OpenAI** cho 100+ model (LiteLLM / OpenRouter)
- **Fallback & failover:** model lỗi / rate-limit → tự nhảy sang nhóm dự phòng
- **Load-balance nhiều key**, retry/backoff, cost tracking ở một chỗ

Nhớ trần rate-limit do vendor đặt? Router giúp **vượt trần** bằng nhiều key/nhiều provider và sống sót khi một provider hỏng.

Tái dùng KV-cache của **prefix** chung (system prompt, RAG context).

- Anthropic: cache read $\sim 0.1 \times$ ($\sim 90\%$ rẻ hơn), cần khai báo breakpoint
- OpenAI: **tự động** với prompt ≥ 1024 token, $\sim 50\%$ rẻ hơn

Trả lại câu trả lời cũ cho câu hỏi **tương tự về nghĩa** (so embedding).

- GPTCache: lưu embedding query trong Redis
- Tránh gọi LLM lặp $\rightarrow$ cắt cả cost lẫn latency
- Cảnh thận: trả lời cũ có thể lỗi thời

Lưu ý: Hai tầng cache khác bản chất: **prompt cache** (provider) so khớp **prefix giống hệt** từng byte $\rightarrow$ đặt phần tĩnh lên đầu; **semantic cache** (của bạn) so khớp **ý nghĩa** $\rightarrow$ mạnh hơn nhưng phải canh ngưỡng similarity kéo trả nhằm câu cũ đã lỗi thời.

Frontier-Scale Serving — Các “Ông Lớn” Phục Vụ Thế Nào

Bạn không cần tự xây những thứ này, nhưng hiểu chúng giải thích **tại sao cost & latency như vậy** — và một sợi chỉ đỏ duy nhất xuyên suốt: *giữ GPU bận, đừng phí KV cache*

Static batching

Cả batch chờ request **dài nhất** xong → GPU ngồi không khi các request ngắn đã xong.

Continuous (in-flight) batching

Lên lịch lại batch **mỗi bước decode**: request xong thì thay ngay request mới vào.

Orca (OSDI'22) giới thiệu iteration-level scheduling, báo cáo tới **36.9×** throughput so với baseline ở cùng latency. Đây là “unlock” lớn nhất của GPU utilization khi serve LLM.

Lưu ý: Mọi kỹ thuật phía sau đều phục vụ một ý: **không tính lại / không phí KV cache, và GPU không bao giờ rảnh.**

Mỗi token sinh ra cần lưu Key/Value (KV cache). Cấp phát liên mạch → phân mảnh, phí bộ nhớ. **PagedAttention** (vLLM, SOSP'23) chia KV cache thành **block cố định, không liên mạch** — y như virtual memory paging của hệ điều hành.

- Gần **zero** lãng phí KV → batch lớn hơn trong cùng VRAM
- Chia sẻ KV giữa request (parallel sampling, beam) qua copy-on-write
- vLLM báo cáo **2–4×** throughput so với hệ trước ở cùng latency

KV cache là lý do context dài **tốn bộ nhớ** và vì sao prompt caching (slide trước) tiết kiệm được — nó tái dùng đúng những block KV này.

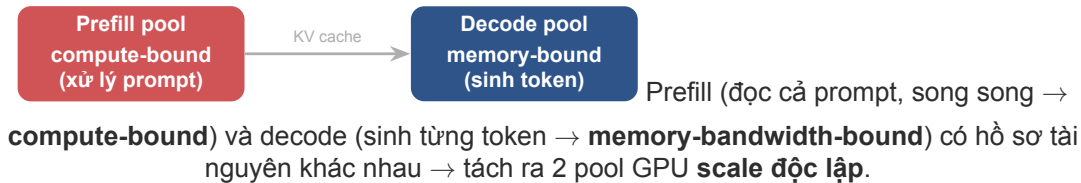
Prefill của prefix chung tính **một lần**, dùng lại cho nhiều request.

- Provider phôi ra dưới dạng giảm giá input: Anthropic $\sim 90\%$, OpenAI $\sim 50\%$, DeepSeek $\sim 10\times$
- Đặt phần **tĩnh** (system, few-shot, RAG) lên **đầu** prompt

Model **nháp** nhỏ đề xuất 5–8 token, model **đích** xác minh song song.

- Tận dụng GPU đang rảnh, **không đổi** phân phối output
- $\sim 2\text{--}3\times$ giảm latency; EAGLE-3 báo cáo tới $\sim 4.8\times$ (Llama-3.3-70B)

Hai kỹ thuật này là lý do cùng một câu hỏi gửi lần 2 (cache hit) **re và nhanh hơn hẳn** — thiết kế prompt để tận dụng.



- DistServe (OSDI'24): tới **7.4×** nhiều request hơn trong cùng SLO
- Mooncake (Kimi, FAST'25): KV-cache-centric, >100B token/ngày
- NVIDIA Dynamo (GTC 3/2025): prefill/decode là first-class, KV-aware router

Lab	Hạ tầng (mốc công khai)
OpenAI	Stargate \$500B/4 năm (1/2025); Azure + Oracle + CoreWeave + Google + AWS (đa cloud từ 2025)
Anthropic	AWS Trainium (Project Rainier, ~500k chip) + Google TPU (tới 1M) + NVIDIA — đa silicon
Google	TPU Ironwood (v7, 4/2025, chuyên inference); serve Gemini trên silicon nhà
Meta	Llama open-weight ; train Llama 4 trên cụm >100k H100 (báo chí)

Lưu ý: GPU economics chi phối: thiếu hụt H100/H200 (lead time ~36–52 tuần), giá thuê tăng; reasoning model ăn thêm inference-compute → **GPU-hour** (không phải CPU) quyết định cost serve agent. (Số tiền/giá = “reported”).

CI/CD & Eval Gates

Deploy bằng tay sẽ quên bước, sẽ sai. Tự động hoá
build→push→deploy — và thêm lớp **eval gate** riêng của AI: chặn
deploy nếu chất lượng tụt

12

```
# .github/workflows/deploy.yml
steps:
  - uses: actions/checkout@v4
    # build + push image to GHCR (needs packages: write)
  - uses: docker/build-push-action@v6
    # scan image for CVEs, fail the build on high severity
  - uses: aquasecurity/trivy-action@master
  - run: promptfoo eval --fail-on-error # <- EVAL GATE
  - run: railway up # deploy iff evals pass
```

Agent **non-deterministic** → không thể chỉ dựa unit test exit 0. Gate deploy theo **điểm eval** $\geq$ **ngưỡng** (promptfoo / DeepEval / Braintrust). Đây là câu nổi tới Day 14 (Evaluation).

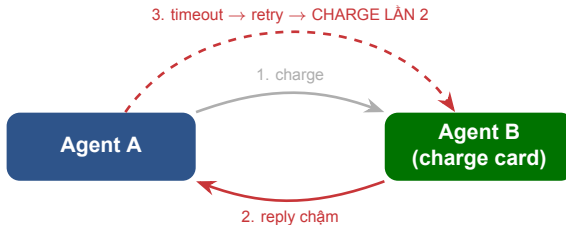
Nâng Cao — Deploy Cấp Production (tùy chọn)

Đưa 1 agent lên URL an toàn mới là 20% dễ. 80% khó là khi agent **bền bỉ, lặp lại, hành động không hoàn tác được, và nói chuyện với nhau** — phần này phác qua những vấn đề đó

Hình dạng	Trigger	Vòng đời	Scale	State
Chatbot đồng bộ	user request	giây (timeout)	per-request	session store
Cron / nền	scheduler	phút	thấp, định kỳ	job state
Batch	dataset/queue	dài, async	fan-out	per-item idempotent
Autonomous “chạy mãi”	loop liên tục	vô hạn	1 actor/goal	phải sống qua restart
Copilot nhúng	in-app event	giây	theo app	app/session
Ambient / sự kiện	webhook/event	bursts, ngủ lâu	scale-to-zero	bền qua giấc ngủ

Lưu ý: State-durability là trục phân biệt chính. “Autonomous chạy mãi” phá vỡ mô hình request/response → buộc dùng durable runtime. Multi-agent: tốn ~15× token; nguyên tắc vàng “**read thì song song được, write thì không**” — scale 1 agent trước, chỉ tách khi chạm trần thật.

Web app thường: lỗi thì retry. Nhưng agent có **side effect thật** (gửi mail, trừ tiền, đặt vé). Retry



mù = làm hai lần.

Lưu ý: Timeout **không phân biệt** được “thất bại” với “thành công nhưng reply chậm” → caller bắn lại side effect. Sự thật phũ phàng: **không có “exactly-once”** cho hành động ngoài hệ thống — chỉ có **at-least-once + consumer idempotent**.

Client gửi header Idempotency-Key (UUID) kèm request mutating.

- Server lưu kết quả lần **đầu** theo key
- Retry cùng key → trả lại y nguyên (kể cả lỗi đã cache)
- Có IETF draft chuẩn hoá header này

LLM **không sinh lại tham số y hệt** cho cùng một ý định → hash nội dung thô **trượt**.

- Cần **dedup theo ngữ nghĩa / ý định**
- “Cùng intent”, không phải “cùng bytes”
- Ai là trọng tài quyết 2 hành động là “một”?

Mọi tool call có side effect nên mang một idempotency key ổn định (vd `order_id`), và backend dedup bằng Redis SET NX + TTL.

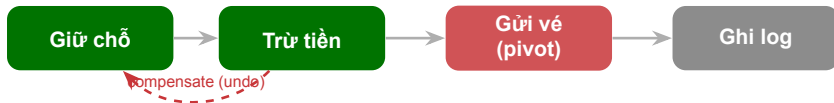
Agent crash ở bước 7 (sau 4 tool call). Chạy lại từ đầu = **trả tiền LLM lần nữa** + lặp tool đã ghi đĩa. Durable execution giải quyết bằng một mẹo tinh tế:

Temporal / Restate / Inngest: ghi **output của LLM** vào journal ở lần đầu; khi replay thì **đọc lại bản ghi, KHÔNG gọi lại model**.

- “Phần thông minh” không bao giờ chạy lại
- Bước đã xong được memoize, bỏ qua

Lưu ý: LangGraph là ngoại lệ: checkpoint ở mức **node**, không phải từng call → node chưa xong sẽ **chạy lại cả LLM call**. “Checkpoints $\neq$ durable execution”.

Lưu ý: “Exactly-once” của các framework thực ra là “*exactly-once trên datastore của họ*”. Hành động ra hệ thống ngoài vẫn cần idempotent. Câu hỏi hay: agent được **resume** có còn “suy nghĩ” không, hay chỉ đang **đọc lại** quá khứ?



- **Saga**: mỗi bước có một bước **bù trừ** (undo) — không có rollback tự động, phải tự code
- Bước **không hoàn tác được** (gửi mail/vé) = **pivot**: đặt **cuối** + gate bằng **human approval**
- Vấn đề: agent tự chọn thứ tự hành động — runtime có nên **cấm** nó xếp việc bất khả hoàn trước pivot?

Lưu ý: HITL nghịch lý: Temporal cho agent **chờ 3 tuần tốn 0 compute** — nhưng giữ chỗ / báo giá / token **vẫn hồng dần** ngoài đời. Resume một quyết định đã cũ = durability thành gánh nặng.

Đây là mặt **hạ tầng** của an toàn (khác Day 11 — mặt hành vi). Tool của agent gọi ra ngoài = kênh rò rỉ.

Rò rỉ data cần 3 thứ **cùng lúc**:

- data riêng tư
- + nội dung không tin cậy
- + kênh gửi ra ngoài

Bỏ một cái = chặn được.

EchoLeak · CamoLeak · GitLab Duo · AgentFlayer — đều **render ảnh/HTML tới URL kẻ tấn công** và đều fix bằng **chặn render / giới hạn egress**.

Lưu ý: Egress allowlist là phòng thủ mạnh nhất — nhưng **kênh rò rỉ thường là kênh tin cậy**: CamoLeak tuồn data qua chính proxy Camo của GitHub. Không thể allowlist khỏi nhà cung cấp bạn đang tin.



cô lập mạnh hơn → (startup chậm hơn, gần như ngược lại)

Lưu ý: CVE-2025-66479 (Claude Code): cấu hình `allowedDomains: []` (ý định *chặt nhất*) lại **fail-open** (mở toang egress) vì code check `length > 0`. Ý định an toàn nhất **không biểu diễn được**. → Ngữ nghĩa mặc định **CHÍNH LÀ** thuộc tính bảo mật (fail-closed, không phải fail-open).

- OWASP **LLM06 Excessive Agency**: quá nhiều quyền → hành động phá hoại
- **Confused deputy**: agent bị lừa dùng quyền của nó cho kẻ khác
- 1 “god-key” chung = 1 điểm sụp đổ

- Mỗi agent một **identity** riêng, scope hẹp
- Token **ngắn hạn**, theo từng task
- Audience-bound token (MCP OAuth 2.1)
- VD: Microsoft Entra Agent ID

Lưu ý: Khoảng cách thực tế: ~91% tổ chức chạy agent ở production, nhưng chỉ ~10% quản chúng **như một danh tính**. Per-agent identity ở quy mô lớn là bài toán vận hành chưa được giải tốt.

Deployment Checklist

Một trang để soi trước khi bấm deploy — gói lại mọi thứ đã học hôm nay

14

- ☐ Multi-stage + uv, <500MB, non-root
- ☐ .dockerignore + scan CVE (Trivy)
- ☐ Public URL + HTTPS hoạt động
- ☐ Env vars (không hardcode secret)

- ☐ Auth (API key / JWT)
- ☐ Rate limit + **spending cap** (admission control)
- ☐ Per-project/per-tenant budget

- ☐ Streaming (SSE) cho response dài
- ☐ State externalized (Redis/Postgres)

- ☐ /health (liveness + readiness)
- ☐ Graceful shutdown (drain)
- ☐ Rollback plan <2 phút
- ☐ **Pin model ID** + tắt auto-upgrade
- ☐ Router/fallback khi provider lỗi

Phụ Lục Thực Hành — Lệnh & Code

Gói lại thành thứ gì được ngay: lệnh deploy thật, và một cost-guard tối thiểu — để rồi lớp học là deploy được

15

A) Google Cloud Run: build from source, then deploy

```
gcloud run deploy agent-svc --source . \
  --port 8080 --concurrency 8 --memory 1Gi \
  --region asia-southeast1 --allow-unauthenticated \
  --set-env-vars MODEL_ID=claude-...,MAX_USD_PER_REQ=0.05 \
  --set-secrets ANTHROPIC_API_KEY=anthropic-key:latest
```

B) Railway: deploy + set environment variables

```
railway up # streams build + deploy logs
railway variables --set "MODEL_ID=..." \
  --set "ANTHROPIC_API_KEY=sk-..."
```

Lưu ý: `--concurrency` để **thấp** cho agent: mỗi request giữ một connection streaming dài. Secret tiêm qua `--set-secrets` / Variables tab — **không bao giờ** nhét vào image.

```
MAX_USD = float(os.environ["MAX_USD_PER_REQ"]) # e.g. 0.05

def guard(messages, model, user_id):
    in_tok = count_tokens(messages, model) # count tokens
    est = in_tok/1e6*IN_PRICE + MAX_OUT/1e6*OUT_PRICE
    if est > MAX_USD: # admission control
        raise BudgetExceeded(f"${est:.3f} > ${MAX_USD}")
    metrics.tag(user=user_id, feature="chat", usd=est)
    return est
```

Tag mọi call, đếm token có thẩm quyền, **ép budget trước khi gọi LLM**. Nhớ: alert của provider là phản ứng *sau*; admission control chặn *trước*.

Hands-on & Key Takeaways

Mục tiêu cuối cùng rất cụ thể: agent có public URL, ai cũng truy cập được, có health check, có basic auth, có cost guard

16

Đóng gói agent thành container, deploy lên cloud, và có public URL hoạt động.

1. Viết Dockerfile (multi-stage + uv, slim base, non-root, <500MB)
2. Build & test container locally: `docker build` → `docker run` → scan bằng Trivy
3. Thêm streaming endpoint (SSE) + health check `GET /health`
4. Deploy lên Railway hoặc Render: connect repo → set env vars → deploy
5. Thêm basic auth (API key) + một rate limit / spending guard đơn giản
6. Demo: gửi request tới public URL, nhận response streaming từ agent

Container

- Dockerfile (multi-stage, uv, <500MB)
- docker-compose.yml + .dockerignore
- Health check + streaming endpoint
- Trivy scan sạch (no high CVE)

Deployment

- Public URL hoạt động (HTTPS)
- Env vars đúng cách (không hardcode)
- Basic auth (API key) + cost guard
- Demo request/response streaming

Lưu ý: Không cần enterprise-grade. Điều cần chứng minh là bạn **biết cách đưa agent từ localhost lên cloud**, nó hoạt động, và **không đốt tiền**.

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Cùng cỗ máy, khác cái hộp:** ship như thường (CI/CD, canary, IaC) — định nghĩa lại **the test** (eval gate), **the bill** (token), **the dependency** (model).
- 2 **Container 2026:** multi-stage + uv + slim/distroless (tránh Alpine), non-root, scan CVE, <500MB; SSE + externalize state.
- 3 **Chạy ở đâu = quyết định deploy:** server-side mặc định; “client-side” cần BFF proxy; chỉ on-device/in-browser mới keyless.
- 4 **Platform theo timeout, scale theo concurrency:** router (LiteLLM) failover + vượt rate-limit; prompt/semantic cache cắt cost.
- 5 **Security & cost:** budget KHÔNG tự chặn → admission control + eval gate trong CI.

Monitoring, Logging & Observability

“Agent deploy xong, 3 ngày sau: latency tăng gấp đôi, cost tăng 300%. Bạn không biết cho đến khi user phàn nàn. ”

- Đọc trước: LangSmith hoặc Langfuse quickstart (20 phút)
- Chuẩn bị: agent deployed từ Lab 12 cần có endpoint để gắn monitoring
- Suy nghĩ: metrics nào quan trọng nhất cho AI agent trên production?

1. Astral, *Using uv in Docker* — docs.astral.sh/uv/guides/integration/docker/. Multi-stage build hiện đại cho Python.
2. Google Cloud, *Cloud Run request timeout & concurrency* — cloud.google.com/run/docs. 60 phút max, concurrency 80/1000.
3. Model Context Protocol, *Transports & Authorization (2025-06-18)* — modelcontextprotocol.io. Streamable HTTP + OAuth 2.1.
4. OWASP, *Top 10 for LLM Applications 2025* — genai.owasp.org/llm-top-10/. LLM10 Unbounded Consumption.
5. Adam Wiggins, *The Twelve-Factor App* — 12factor.net. Factor VI (stateless processes).

Hỏi & Đáp

Từ hôm nay, agent không còn chỉ chạy trên máy bạn. Nó đã là một service thật sự — có URL, có bảo vệ, và không đốt sạch ngân sách.